TRABALHO SEMESTRAL | ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Processo de QA

Qualidade de Software

Git Flow • Registro de bugs • Casos de teste

RESPONSÁVEL

Danilo Penafiel Polonio

VERSÃO

1.0

ÁREA

Quality Assurance (QA)

DATA

Outubro de 2026

Sumário

1. Objetivo 3

2. Fluxo de branches — Git Flow 3

3. Modelo de abertura de bug 5

4. Modelo de caso de teste 7

5. Aplicação nas sprints 8

1. Objetivo

Este documento padroniza o trabalho de qualidade de software (QA) do projeto. Ele define **o fluxo de branches no Git**, **a abertura e a documentação de bugs** e **o formato dos casos de teste** que serão utilizados nas próximas sprints.

A ideia é que todo o grupo siga as mesmas regras e os mesmos modelos, de modo que os problemas encontrados sejam registrados de forma clara, os testes possam ser repetidos por qualquer integrante e as alterações no código cheguem à versão final de maneira organizada.

Tema	O que o documento define	Seção
Git Flow	Branches utilizadas, fluxo de trabalho e regras do grupo	2
Bugs	Conceito e modelo reutilizável para abertura de bugs	3
Casos de teste	Conceito e modelo reutilizável para casos de teste	4
Sprints	Como o processo será aplicado ao longo do projeto	5

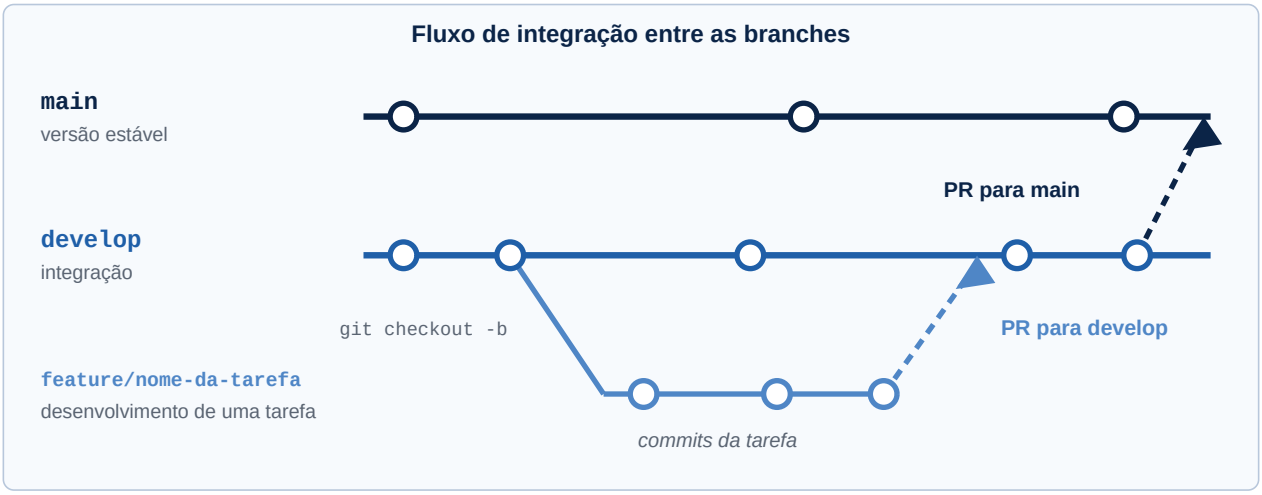
2. Fluxo de branches — Git Flow

O projeto utiliza um fluxo de branches inspirado no Git Flow, em que as funcionalidades são desenvolvidas em branches `feature`, integradas à `develop` por Pull Request (PR) e, depois de validadas, entregues na `main` também por PR.

2.1 Branches utilizadas

Branch	Função	Duração
`main`	Versão estável do projeto, validada para entrega.	Permanente
`develop`	Branch de integração das funcionalidades em desenvolvimento.	Permanente
`feature/nome-da-tarefa`	Branch temporária para implementar uma tarefa específica.	Temporária

2.2 Visão geral do fluxo



Leitura do diagrama: `main` guarda a versão estável; `develop` concentra a integração; cada `feature/nome-da-tarefa` nasce a partir da `develop` para o desenvolvimento de uma única tarefa e volta para ela por Pull Request. Quando uma versão está validada, a `develop` é integrada à `main`, também por Pull Request.

2.3 Fluxo de trabalho passo a passo

Nº	Etapa	Comandos de referência
1	Atualizar a branch `develop`.	`git checkout develop` `git pull origin develop`
2	Criar uma branch `feature/nome-da-tarefa` a partir de `develop`.	`git checkout -b feature/nome-da-tarefa`
3	Desenvolver a funcionalidade e realizar verificações locais.	`git add .` `git commit -m "descrição clara"`
4	Enviar a branch para o repositório remoto.	`git push -u origin feature/nome-da-tarefa`
5	Abrir um Pull Request (PR) direcionado à branch `develop`.	*Realizado na plataforma do repositório remoto.*
6	Solicitar revisão e realizar as correções necessárias.	*Realizado na plataforma do repositório remoto.*
7	Após aprovação, fazer o merge do PR.	*Realizado na plataforma do repositório remoto.*
8	Quando uma versão estiver validada para entrega, integrar `develop` à `main` por meio de Pull Request.	*Realizado na plataforma do repositório remoto.*

Os comandos são apenas uma referência para as etapas que envolvem o Git na linha de comando; o grupo pode usar a ferramenta de sua preferência (terminal, editor ou interface gráfica), desde que o fluxo seja respeitado.

2.4 Regras do grupo

- Não realizar commits diretamente na `main`.
- Toda integração entre branches deve ocorrer por Pull Request.
- Utilizar nomes claros e relacionados à tarefa nas branches.
- Descrever no PR o que foi alterado e como a alteração foi verificada.
- Resolver conflitos antes de concluir o merge.

2.5 Exemplo de nomes de branch

Os nomes devem descrever a tarefa. Exemplos ilustrativos do padrão:

```
feature/tela-de-cadastro
feature/validacao-de-email
```

3. Modelo de abertura de bug

Um **bug** é um comportamento incorreto ou diferente do esperado, identificado durante o desenvolvimento ou os testes. Registrá-lo de forma padronizada permite que quem for corrigir consiga reproduzir o problema sem precisar de explicações adicionais.

3.1 Template reutilizável

Campo	Como preencher
Título	*Resumo curto e objetivo do problema.*
Descrição	*Contexto do defeito.*
Passos para reproduzir	*Sequência numerada de ações necessárias para reproduzir o problema.* *1. …* *2. …* *3. …*
Resultado esperado	*Comportamento correto do sistema.*
Resultado obtido	*Comportamento observado.*
Evidência	*Print de tela ou vídeo, quando aplicável.*
Responsável	*Pessoa encarregada de analisar ou corrigir o problema, quando definida.*
Prioridade	*Urgente, alta, média ou baixa, conforme o impacto.*

3.2 Exemplo preenchido

Campo	Exemplo
Título	Cadastro permite continuar sem preencher o e-mail.
Descrição	Na tela de cadastro, o e-mail é um dado necessário, mas o sistema permite concluir o cadastro sem preenchê-lo.
Passos para reproduzir	1. Abrir a tela de cadastro. 2. Preencher os demais campos. 3. Deixar o e-mail vazio e clicar em Cadastrar.
Resultado esperado	O sistema deve informar que o e-mail é obrigatório e impedir o cadastro.
Resultado obtido	O sistema conclui o cadastro sem e-mail.
Evidência	Anexar uma captura de tela do resultado.
Responsável	A definir.
Prioridade	Média.

Nota: o exemplo é ilustrativo; responsável e prioridade devem ser definidos pelo grupo para cada bug real.

3.3 Orientação para a prioridade

A prioridade deve refletir o impacto do bug no uso do sistema. Como sugestão de referência para o grupo:

Prioridade	Quando usar (sugestão)
Urgente	Impede o uso de uma funcionalidade essencial ou compromete a entrega da versão.
Alta	Afeta uma funcionalidade importante, mas existe alternativa temporária.
Média	Comportamento incorreto que não impede o uso principal do sistema.
Baixa	Problema pequeno ou visual, sem impacto no funcionamento.

4. Modelo de caso de teste

Os **casos de teste** documentam verificações que podem ser executadas de forma repetível para confirmar se uma funcionalidade funciona conforme os requisitos. Com eles, qualquer integrante do grupo consegue executar o mesmo teste e comparar o resultado com o esperado.

4.1 Template reutilizável

Campo	Como preencher
ID	*Identificador único, por exemplo, CT-001.*
Nome	*Objetivo do teste.*
Funcionalidade	*Parte do sistema que será verificada.*
Pré-condições	*Condições necessárias antes de executar o teste.*
Passos	*Sequência de ações.* *1. …* *2. …* *3. …*
Resultado esperado	*Comportamento correto esperado.*
Resultado obtido	*Comportamento observado.*
Status	*Aprovado, reprovado ou bloqueado.*
Responsável	*Pessoa que executou o teste.*

4.2 Exemplo preenchido

Campo	Exemplo
ID	CT-001
Nome	Validar cadastro com campos obrigatórios preenchidos.
Funcionalidade	Cadastro de usuário.
Pré-condições	Tela de cadastro disponível e e-mail ainda não utilizado.
Passos	1. Abrir a tela de cadastro. 2. Preencher os campos obrigatórios com dados válidos. 3. Clicar em Cadastrar.
Resultado esperado	O usuário é cadastrado e recebe uma confirmação.
Resultado obtido	Preencher após executar o teste.
Status	Preencher após executar o teste.
Responsável	Preencher com quem executou o teste.

4.3 Significado dos status

Status	Significado
Aprovado	O resultado obtido é igual ao resultado esperado.
Reprovado	O resultado obtido difere do esperado. Deve-se registrar um bug (seção 3).
Bloqueado	O teste não pôde ser executado por algum impedimento, como funcionalidade indisponível ou outro bug.

5. Aplicação nas sprints

Em cada sprint, o processo de QA será aplicado seguindo estas etapas:

Nº	Etapa	Descrição
1	**Revisar os requisitos**	Revisar os requisitos das funcionalidades.
2	**Definir os casos de teste**	Definir e registrar os casos de teste.
3	**Executar os testes**	Executar os testes quando as funcionalidades estiverem disponíveis.
4	**Registrar bugs**	Registrar bugs quando houver divergências entre o esperado e o observado.
5	**Reexecutar os testes**	Reexecutar os testes após as correções.
6	**Manter a documentação**	Manter a documentação acessível e atualizada no repositório do grupo.